TRAFFIC VIOLATION PENALTY CALCULATOR

Python Activity Record

1. Problem Statement

Develop a Python program that accepts a vehicle type, traffic offence, whether the offence is repeated, and the total number of violations. The program validates the entered values, calculates a sample fine using a predefined fine dictionary, doubles the fine for a repeat offence, and displays a case status based on the total violation count.

2. Objectives

- Accept vehicle type, offence, repeat-offence status, and total violation count from the user.
- Use a dictionary to store sample fines for speeding, helmet, and signal offences.
- Validate the vehicle, offence, repeat status, and non-negative violation count.
- Double the selected sample fine when the repeat-offence answer is yes.
- Show Review required when the total violation count is 3 or more; otherwise show Normal review.
- Display the vehicle, offence, sample fine, and case status.

3. Assumptions

- Vehicle must be bike or car.
- Offence must be speeding, helmet, or signal.
- Repeat-offence input must be yes or no.
- Total violations must be zero or greater and must be entered as an integer.
- Sample fines are speeding = Rs. 1000, helmet = Rs. 500, and signal = Rs. 1500.
- A repeated offence doubles the base sample fine.
- A count of 3 or more produces Review required; a count below 3 produces Normal review.
- The fines are sample values defined in the program, not confirmation of current official legal penalties.

4. Algorithm

1. Start.

2. Input vehicle, offence, repeat status, and total violation count.

3. Create/use the fine dictionary for the three offence types.

4. Check whether the vehicle, offence, repeat status, and count are valid.

5. If any check fails, display Invalid input.

6. Otherwise, select the base fine for the offence.

7. If repeat status is yes, double the base fine; otherwise keep it unchanged.

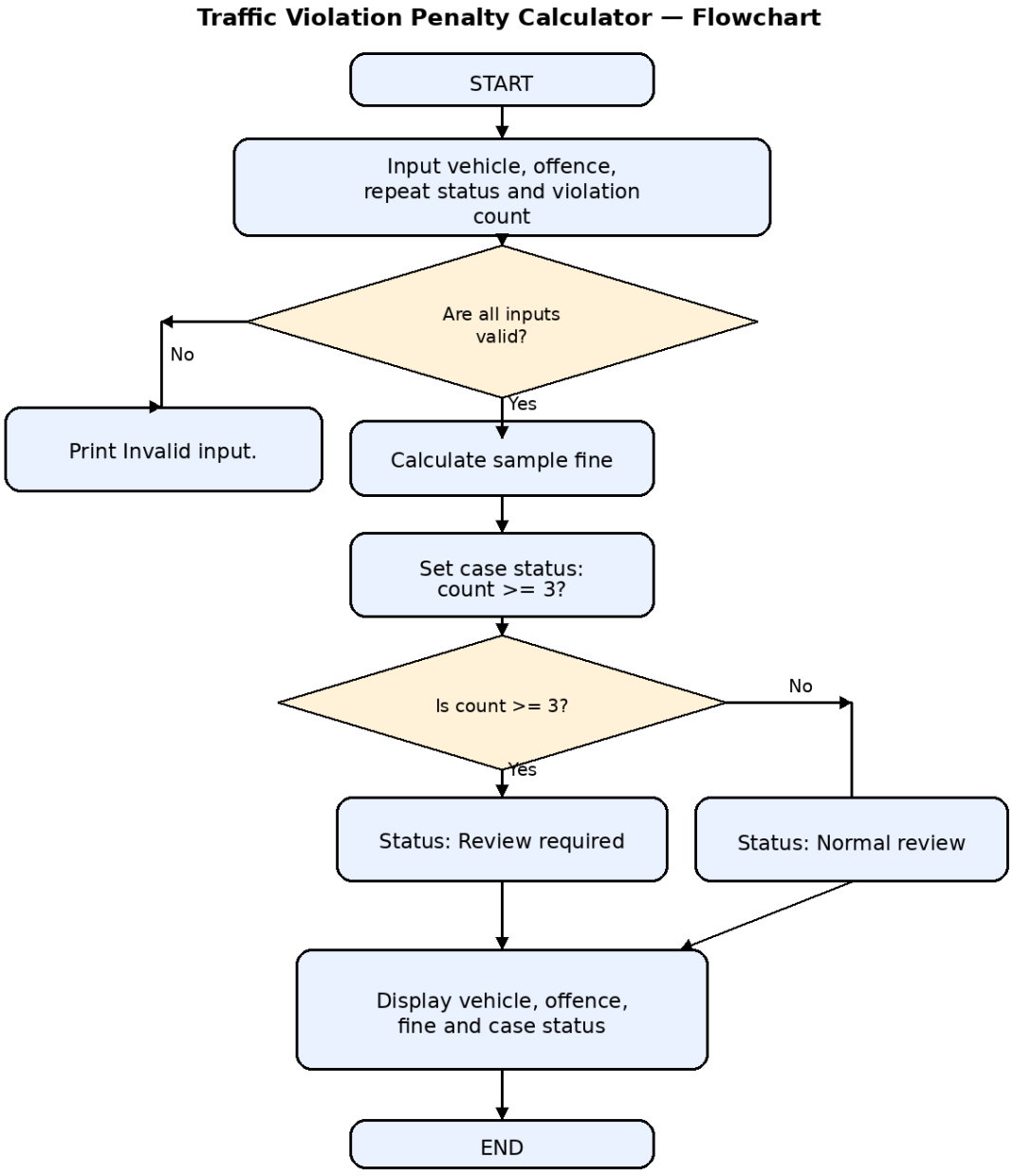
8. If total violations are 3 or more, set case status to Review required; otherwise set it to Normal review.

9. Display the vehicle, offence, sample fine, and case status.

10. End.

5. Flowchart

The flowchart represents the validation, fine calculation, and case-status decision.



6. Pseudocode

```
BEGIN
  INPUT vehicle
  INPUT offence
  INPUT repeat
  INPUT count
```

```
  fines = {speeding: 1000, helmet: 500, signal: 1500}

  IF vehicle not in [bike, car]
    OR offence not in fines
    OR repeat not in [yes, no]
    OR count < 0 THEN
     DISPLAY "Invalid input."
  ELSE
    fine = fines[offence]
    IF repeat = yes THEN
      fine = fine * 2
    END IF

    IF count >= 3 THEN
      status = "Review required"
    ELSE
      status = "Normal review"
    END IF

    DISPLAY vehicle, offence, fine, status
  END IF
END
```

7. Python Source Code

```
vehicle = input("Vehicle (bike/car): ").lower()

offence = input("Offence (speeding/helmet/signal): ").lower()

repeat = input("Repeat offence? (yes/no): ").lower()

count = int(input("Total violations: "))

fines = {"speeding": 1000, "helmet": 500, "signal": 1500}

if vehicle not in ("bike", "car") or offence not in fines or repeat not in ("yes", "no") or count <
0:

    print("Invalid input.")

else:

    fine = fines[offence] * (2 if repeat == "yes" else 1)

    status = "Review required" if count >= 3 else "Normal review"

    print("Vehicle:", vehicle.title())

    print("Offence:", offence.title())

    print("Sample fine: Rs.", fine)

    print("Case status:", status)
```

8. Test Cases

Test	Input	Expected result	Purpose
1	bike, helmet, yes, 3	Fine Rs. 1000; Review required	Repeated offence and count threshold
2	truck, speeding, no, 1	Invalid input.	Invalid vehicle
3	car, signal, no, 1	Fine Rs. 1500; Normal review	Valid non-repeat offence
4	car, speeding, no, -1	Invalid input.	Negative count
5	bike, speeding, no, 0	Fine Rs. 1000; Normal review	Zero count accepted
6	bike, speeding, maybe, 1	Invalid input.	Invalid repeat status

9. Output Screenshots

These sample console screenshots are prepared from the supplied program's logic.

9.1 Repeated offence — review required

Sample output — repeated offence

```
Vehicle (bike/car): bike
Offence (speeding/helmet/signal): helmet
Repeat offence? (yes/no): yes
Total violations: 3
Vehicle: Bike
Offence: Helmet
Sample fine: Rs. 1000
Case status: Review required
```

9.2 Invalid vehicle

Sample output — invalid vehicle

```
Vehicle (bike/car): truck
Offence (speeding/helmet/signal): speeding
Repeat offence? (yes/no): no
Total violations: 1
Invalid input.
```

9.3 Normal review

Sample output — normal review

```
Vehicle (bike/car): car
Offence (speeding/helmet/signal): signal
Repeat offence? (yes/no): no
Total violations: 1
Vehicle: Car
Offence: Signal
Sample fine: Rs. 1500
Case status: Normal review
```

9.4 Negative violation count

Sample output — negative count

```
Vehicle (bike/car): car
Offence (speeding/helmet/signal): speeding
Repeat offence? (yes/no): no
Total violations: -1
Invalid input.
```

10. Conclusion

The program demonstrates input handling, dictionary use, conditional statements, validation, and basic calculation in Python. It chooses a sample fine according to the offence, doubles it for repeat offences, and assigns a case status based on the violation count. The program is educational and does not verify official penalties or connect to a real traffic authority database.